



Урок 4. Сложные запросы

[Инструкция DISTINCT](#)

[Объединение запросов \(UNION и UNION ALL\)](#)

[Объединение таблиц \(JOIN\)](#)

[Основные виды JOIN](#)

[Специальные виды JOIN](#)

[Диаграммы Венна](#)

[Вопросы на собеседовании](#)

[Резюме](#)

Инструкция DISTINCT

Инструкция `DISTINCT` используется для удаления дублирующихся строк из результата выполнения запроса. Применяя `DISTINCT`, мы можем получить только уникальные строки из таблицы. Например, запрос ниже выведет уникальные значения в поле `column1`:

```
SELECT DISTINCT column1 FROM table_name;
```

В случае, когда нам необходимы уникальные комбинации значений в двух и более полях, мы указываем поля через запятую, а инструкцию `DISTINCT` указываем один раз, также после `SELECT`:

```
SELECT DISTINCT column1, column2 FROM table_name;
```

Когда мы используем `DISTINCT`, СУБД запоминает каждое уникальное значение и сравнивает его с последующими, что может замедлить выполнение запроса. Это может быть полезно, например, в случае сетевых ошибок, когда источник данных может отправить для записи в СУБД одно и то же сообщение дважды.

Пример использования инструкции `DISTINCT` :

```
SELECT DISTINCT type
FROM locations;
```

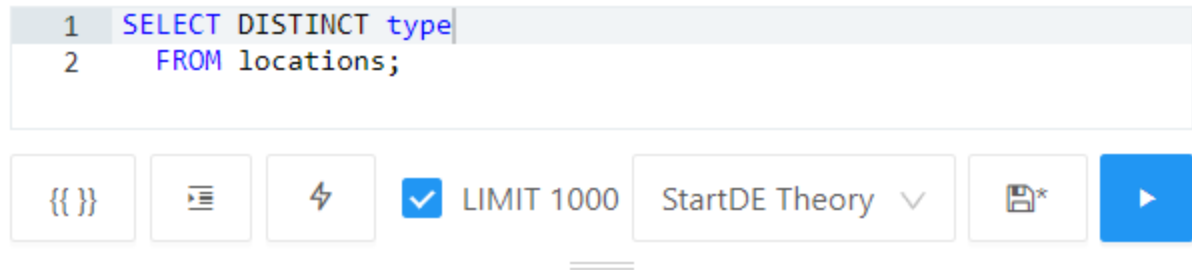
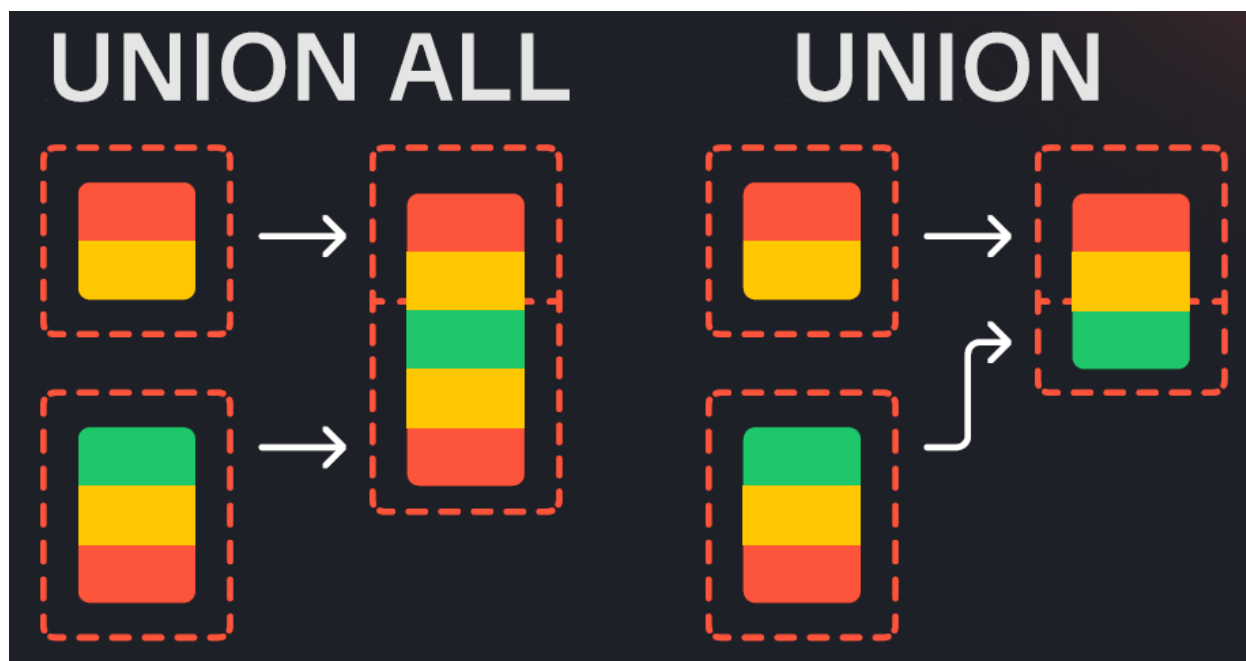


Table	+
type	
Non-Diegetic Alternative Reality	
Death Star	
Arcade	

Объединение запросов (`UNION` и `UNION ALL`)

Операции `UNION` и `UNION ALL` используются для объединения результатов двух и более запросов:

- `UNION` объединяет результаты, исключая дублирующиеся строки;
- `UNION ALL` работает аналогично `UNION`, но не исключает дубликаты.



Таким образом, `UNION` (в отличие от `UNION ALL`) выполняет дедупликацию.



Дедупликация — исключение повторяющихся данных

Дедупликация требует дополнительных ресурсов и времени на выполнение, поэтому `UNION ALL` работает быстрее, так как он просто объединяет результаты без удаления дубликатов.

Общий вид SQL-запроса с использованием UNION:

```
SELECT <поле_1>, <поле_2>, <поле_3>      -- запрос на выборку из:
FROM table_1
UNION [ALL]                               -- инструкция для объеди
SELECT <поле_1>, <поле_2>, <поле_3>      -- запрос на выборку из:
FROM table_2;
```

У операций `UNION` и `UNION ALL` есть ограничение: **количество полей** в объединяемых результатах запросов **должно быть одинаковым**. При этом названия полей могут различаться, так как итоговые названия полей берутся из первого запроса. Типы данных в соответствующих полях также могут

отличаться, но рекомендуется объединять только поля с совместимыми типами данных.

Пример запроса с использованием UNION:

```
SELECT name  
  FROM locations  
  
UNION ALL  
  
SELECT name  
  FROM characters  
ORDER BY 1;
```

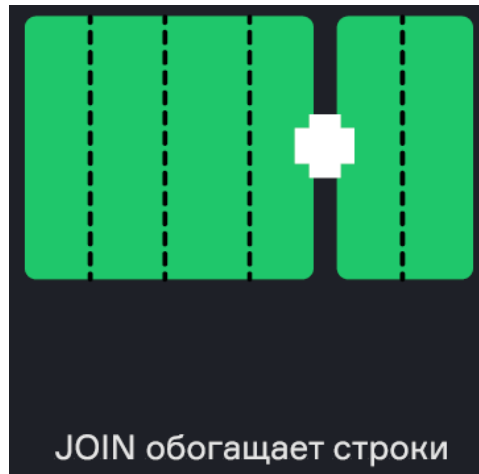
```
1 SELECT name
2   FROM locations
3
4 UNION ALL
5
6 SELECT name
7   FROM characters
8 ORDER BY 1;
```

SQL query editor interface showing a UNION ALL query. The query selects names from 'locations' and 'characters' tables, ordered by the first column. The interface includes a toolbar with icons for syntax highlighting, formatting, execution, and a 'LIMIT 1000' checkbox. A dropdown menu shows 'StartDE Theory'. A blue play button is on the right.

Table +

name
26 Years Old Morty
40 Years Old Morty
7+7 Years Old Morty
80's snake

Объединение таблиц (JOIN)



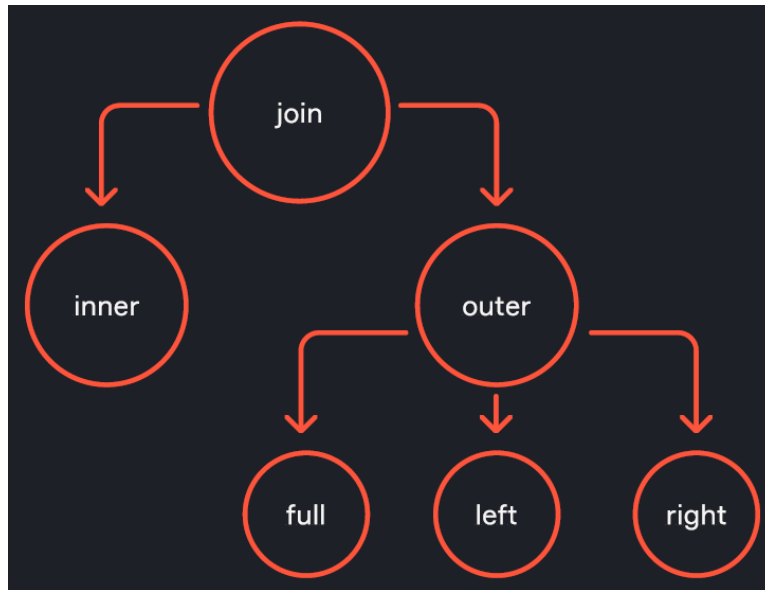
Операция `JOIN` используется для обогащения данных из одной таблицы данными из другой. При этом каждой строке в одной таблице подбирается соответствие во второй таблице по некоторому условию, затем происходит объединение. Таким условием, например, является равенство значений в определенном поле (или полях), когда во второй таблице ищутся записи, соответствующие значению поля из первой таблицы, после чего происходит их объединение в результирующую таблицу.

Общий вид SQL-запроса с использованием JOIN:

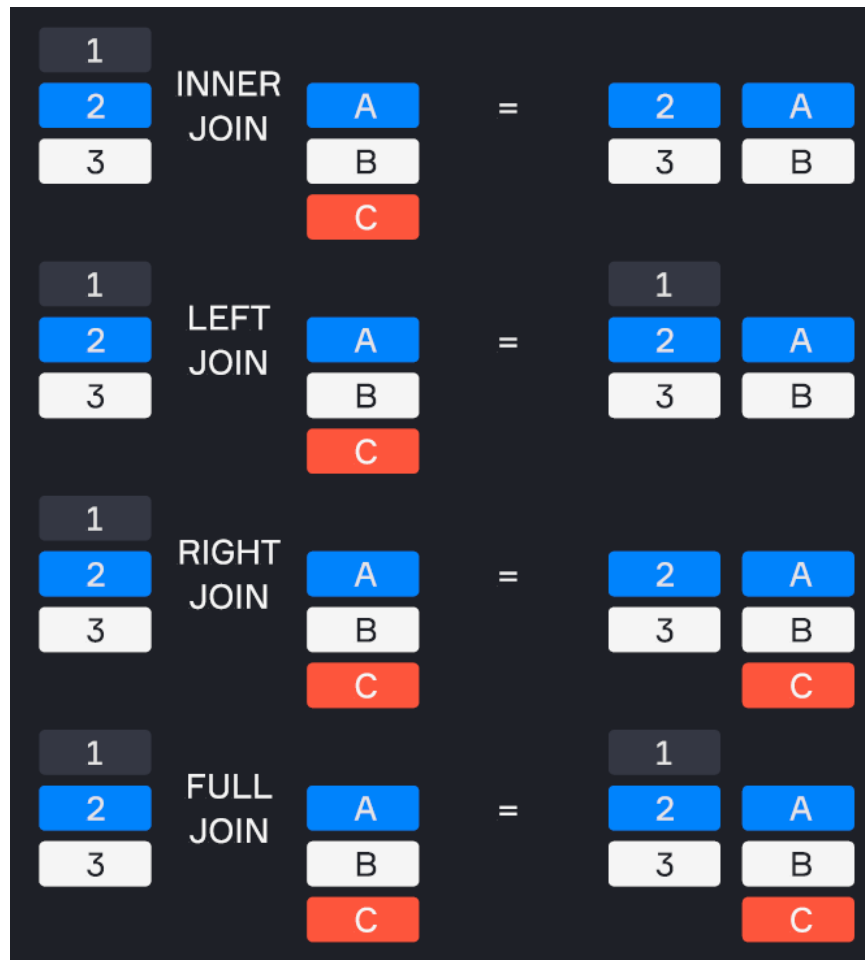
```
SELECT a.column1, b.column2
FROM table1 a
JOIN table2 b ON a.id = b.table2_id;
```

Этот запрос объединяет таблицы `table1` и `table2` по полю `id` из `table1` и полю `table2_id` из `table2`. Обратите внимание, что при объединении таблицам можно присваивать алиасы, чтобы не писать название таблиц. При этом допустимо не использовать инструкцию `AS`.

Основные виды JOIN



Существует несколько типов операций `JOIN`, каждая из которых имеет свои особенности и применяется в зависимости от задачи. В SQL при использовании оператора `JOIN` слово `INNER` можно опустить, поскольку по умолчанию выполняется именно внутреннее соединение (`INNER JOIN`). При использовании операторов `LEFT JOIN` и `RIGHT JOIN` слово `OUTER` также можно опустить. Это упрощает синтаксис и делает запросы более читаемыми, не изменяя их смысл.



В качестве примера возьмем таблицу сотрудников `employees` с полями `employee_id`, `name` и `department_id`. И таблицу департаментов `departments` с полями `department_id` и `department_name`.

employee_id	name	department_id	department_id	department_name
1	John	1	1	HR
2	Jane	2	2	Engineering
3	Mike	4	3	Sales
4	Anna	3		
5	Steve	NULL		
6	Bob	2		

INNER JOIN (внутреннее соединение): возвращает строки, у которых есть соответствие в обеих таблицах.

Для того, чтобы найти только тех сотрудников, для которых в базе данных есть информация об их отделах, воспользуемся **INNER JOIN** :

```
SELECT e.employee_id, e.name, d.department_name
FROM employees e
INNER JOIN departments d ON e.department_id = d.department_id;
```

В результат будут включены только те сотрудники, у которых есть соответствующий **department_id** в таблице **departments** . Сотрудники, у которых **department_id** равен **NULL** или не имеют соответствия в таблице **departments** , будут исключены из результата.

employee_id	name	department_name
1	John	HR
2	Jane	Engineering
4	Anna	Sales
6	Bob	Engineering

OUTER JOIN (внешнее соединение):

- **LEFT JOIN (LEFT OUTER JOIN)**: возвращает все строки из левой таблицы и соответствующие строки из правой таблицы. Если соответствий нет, то вместо значений правой таблицы будут **NULL** .

Для того, чтобы обогатить таблицу сотрудников информацией о названиях отделов воспользуемся **LEFT JOIN** :

```
SELECT e.employee_id, e.name, d.department_name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.department_id;
```

В результат будут включены все сотрудники из таблицы **employees** . Для сотрудников, у которых есть соответствие в таблице **departments** , будет проставлено соответствующее название департамента. Если такого соответствия нет, то **NULL** .

employee_id	name	department_name
1	John	HR
2	Jane	Engineering
3	Mike	NULL
4	Anna	Sales
5	Steve	NULL
6	Bob	Engineering

- **RIGHT JOIN (RIGHT OUTER JOIN):** возвращает все строки из правой таблицы и соответствующие строки из левой таблицы. Если соответствий нет, то вместо значений левой таблицы будут **NULL**.

Для того, чтобы обогатить таблицу отделов информацией о сотрудниках, воспользуемся **RIGHT JOIN**:

```
SELECT e.employee_id, e.name, d.department_name
FROM employees e
RIGHT JOIN departments d ON e.department_id = d.department_id;
```

Запрос позволяет увидеть сотрудников, у которых указан департамент и он существует в таблице департаментов. Также он покажет, какие департаменты существуют, даже если в них нет сотрудников.

employee_id	name	department_name
1	John	HR
2	Jane	Engineering
4	Anna	Sales
6	Bob	Engineering

- **FULL JOIN (FULL OUTER JOIN):** возвращает все строки из обеих таблиц, заполняя отсутствующие значения **NULL**.

```
SELECT e.employee_id, e.name, d.department_name
FROM employees e
FULL JOIN departments d ON e.department_id = d.department_id;
```

Запрос позволяет полностью объединить информацию о сотрудниках и департаментах. В данном случае результат запроса будет совпадать с результатом `LEFT JOIN`, так как все строки из левой таблицы (`employees`) присутствуют в объединении и все строки из правой таблицы (`departments`) имеют соответствия в левой таблице, так что дополнительные строки с `NULL` не добавляются в результат.

employee_id	name	department_name
1	John	HR
2	Jane	Engineering
3	Mike	NULL
4	Anna	Sales
5	Steve	NULL
6	Bob	Engineering

Еще один пример работы FULL JOIN:

id	name	description
1	Data Engineer	Обучение дата-инженеров
2	Data Analyst	Обучение дата-аналитиков
3	Start ML	Начальный курс по ML

id	course_id	number	start_date
1	1	31	2024-03-07
2	1	32	2024-04-01
3	1	33	2024-04-21

Соединение таблицы курсов и потоков:

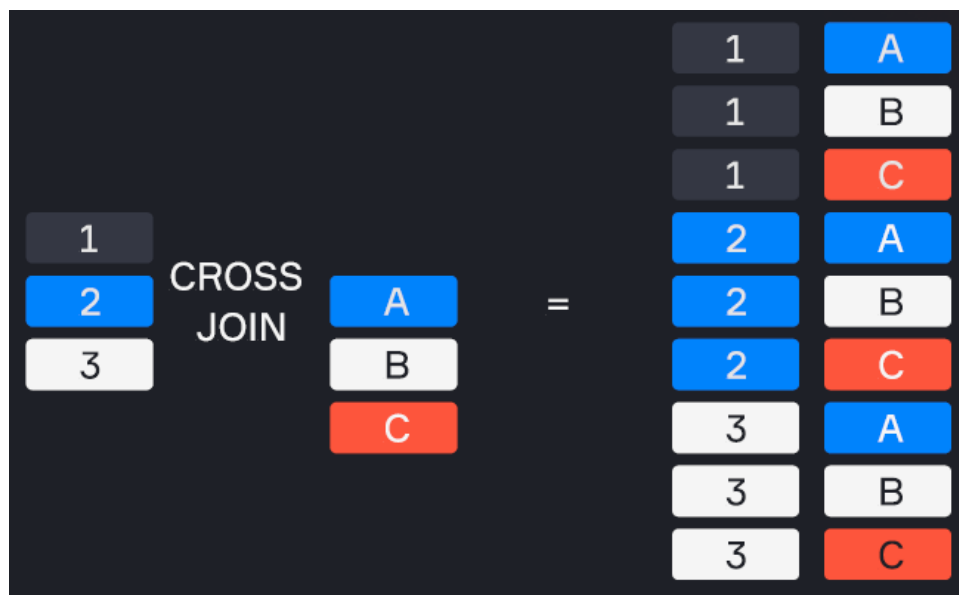
```
SELECT c.id AS course_id,
       c.name AS course_name,
       f.number AS flow_num,
       f.start_date AS flow_start
FROM courses AS c
FULL JOIN flows AS f
ON c.id = f.course_id
```

Таблица `courses` соединяется с таблицей `flows` с помощью `FULL JOIN`, при этом значения из таблицы `courses`, где отсутствуют соответствия, автоматически заполняются `NULL`.

course_id	course_name	flow_num	flow_start
1	Data Engineer	31	2024-03-07
1	Data Engineer	32	2024-04-01
1	Data Engineer	33	2024-04-21
2	Data Analyst	NULL	NULL
3	Start ML	NULL	NULL

Специальные виды JOIN

- **CROSS JOIN:** соединение без условия, создающее декартово произведение двух таблиц. Он полезен в ситуациях, когда нужно сопоставить каждую строку одной таблицы с каждой строкой другой таблицы. Однако его следует использовать с осторожностью из-за возможного экспоненциального роста количества строк в результате, что может привести к значительному снижению производительности.



- **SELF JOIN:** соединение таблицы с самой собой. Это полезно, когда необходимо сравнить строки одной и той же таблицы или получить данные, которые находятся в разных строках, но связаны между собой. Self Join часто используется для работы с иерархическими данными,

например, для анализа отношений между сотрудниками и их начальниками в одной таблице.

Пример работы CROSS JOIN:

```
SELECT c1.name AS name_1, c2.name AS name_2
FROM characters AS c1
CROSS JOIN characters AS c2
WHERE c1.name != c2.name;
```

```
1 SELECT c1.name AS name_1, c2.name AS name_2
2 FROM characters AS c1
3 CROSS JOIN characters AS c2
4 WHERE c1.name != c2.name;
```

{{ }}



LIMIT 1000

StartDE Theory ▾



Table



name_1

name_2

Rick Sanchez

Morty Smith

Rick Sanchez

Summer Smith

Rick Sanchez

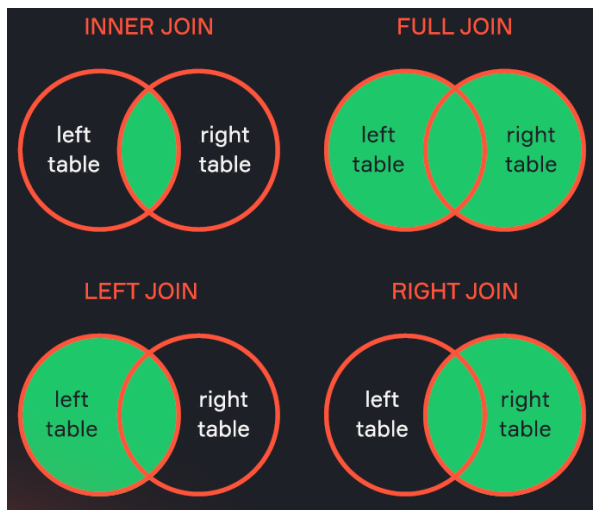
Beth Smith

Rick Sanchez

Jerry Smith

В запросе выше мы выводим все возможные попарные вариации имен.

Диаграммы Венна



Диаграммы Венна часто используются для объяснения различных типов JOINS в SQL, потому что они наглядно демонстрируют пересечения и объединения множеств.

Однако диаграммы Венна не всегда точно передают все аспекты и нюансы операций JOIN в контексте реляционных баз данных (например, если значения в таблице повторяются). Подробнее об этом можно почитать в статьях [Понимание джойнов сломано \(habr.com\)](https://habr.com/ru/articles/444444/) и [Понимание джойнов сломано. Продолжение \(habr.com\)](https://habr.com/ru/articles/444444/).

Вопросы на собеседовании

1. Как DISTINCT влияет на производительность запросов?

DISTINCT увеличивает время выполнения запроса, так как СУБД должна сравнивать и запоминать каждое уникальное значение.

2. Что будет, если выбрать DISTINCT * из таблицы?

Мы получим полную дедупликацию данных.

3. В чем разница между UNION и UNION ALL?

UNION удаляет дубликаты, UNION ALL — нет. UNION ALL работает быстрее, так как не требует дедупликации.

4. Как можно объединить поля разных типов?

Необходимо перевести один тип в другой, например, используя инструкцию CAST.

5. Какие виды JOIN вы знаете?

INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, CROSS JOIN, SELF JOIN.

6. В таблице t1 N-строк, в таблице t2 M-строк. Какой максимум строк мы можем получить при Join?

При CrossJoin мы можем получить $N * M$ строк.

7. Как мы можем выбрать все возможные пары сотрудников, работающих в одном отделе?

Необходимо объединить две таблички и отфильтровать те строки, в которых сотрудник join-ился на самого себя.

8. Как сдвоить таблицы по полям с разными типами данных?

Привести одно из полей к нужному типу с помощью

`CAST` или `::`.

9. Что произойдет, если использовать условие WHERE в CROSS JOIN?

Сначала будет выполнен CROSS JOIN, затем отфильтруются строки по условию WHERE, что может занять больше ресурсов.

Резюме

- Инструкция `DISTINCT` используется для удаления дублирующихся строк из результата запроса
- Операции `UNION` и `UNION ALL` используются для объединения результатов двух запросов:
 - `UNION` объединяет результаты, исключая дублирующиеся строки
 - `UNION ALL` работает аналогично `UNION`, но не исключает дубликаты
- Операция `JOIN` используется для обогащения данных из одной таблицы данными из другой. При этом каждой строке в одной таблице ищется соответствие во второй таблице по некоторому условию или без него. Виды соединений: `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, `FULL JOIN`, `CROSS JOIN`
- `SELF JOIN` — соединение таблицы с самой собой. Это полезно, когда необходимо сравнить строки одной и той же таблицы или получить данные, которые находятся в разных строках, но связаны между собой.

К концу текущего урока вы можете написать SQL-запрос с использованием инструкций:

SELECT	Основной оператор для запроса данных из одной или нескольких таблиц. Позволяет выбрать конкретные столбцы и строки на основе заданных условий.
DISTINCT	Используется вместе с SELECT для удаления дублирующихся строк из результата запроса
FROM	Указывает на таблицу в базе данных, к которой обращается запрос
JOIN	Соединяет строки из двух или более таблиц на основе связанного столбца между ними. Различные типы соединений включают: INNER JOIN, LEFT JOIN (или LEFT OUTER JOIN), RIGHT JOIN (или RIGHT OUTER JOIN), FULL JOIN (или FULL OUTER JOIN), CROSS JOIN
WHERE	Указывает условия, по которым строки будут выбраны
ORDER BY	Упорядочивает строки в результате запроса по одному или нескольким столбцам. Может быть использован для сортировки по возрастанию (ASC) или убыванию (DESC)
LIMIT и OFFSET	LIMIT ограничивает количество возвращаемых строк. OFFSET пропускает указанное количество строк перед возвратом оставшихся строк
UNION и UNION ALL	Объединяют результаты двух или более SELECT запросов. UNION удаляет дубликаты, а UNION ALL сохраняет все строки, включая дубликаты